

Java Down and Dirty

Clojure's Java support is both powerful and lean. It's powerful, in that it brings the expressiveness of Lisp syntax, plus some syntactic sugar tailored to Java. It's lean, in that it can get right to the metal. Clojure code compiles to bytecode and does not have to go through any special translation layer on the way to Java.

Clojure embraces Java and its libraries. Idiomatic Clojure code calls Java libraries directly and does not try to wrap everything under the sun to look like Lisp. This surprises many new Clojure developers but is very pragmatic. Where Java isn't broken, Clojure doesn't fix it.

In this chapter, you will see how Clojure access to Java is convenient, elegant, and fast:

- Calling Java is simple and direct. Clojure provides syntax extensions for accessing anything you could reach from Java code: classes, instances, constructors, methods, and fields. Although you will typically call Java code directly, you can also wrap Java APIs and use them in a more functional style.
- Clojure is *fast*, unlike many other dynamic languages on the JVM. You can use custom support for primitives and arrays, plus type hints, to cause Clojure's compiler to generate the same code that a Java compiler would generate.
- Clojure's exception handling is easy to use. Better yet, explicit exception handling is rarely necessary. Clojure's exception primitives are the same as Java's. However, Clojure does not require you to deal with checked exceptions and makes it easy to clean up resources using the with-open idiom.

9.1 Exception Handling

In Java code, exception handling crops up for three reasons:

- Wrapping checked exceptions (see [Checked Exceptions, on page 205](#) if you are unfamiliar with checked exceptions)
- Using a finally block to clean up nonmemory resources such as file and network handles
- Responding to the problem: ignoring the exception, retrying the operation, converting the exception to a nonexceptional result, and so on

In Clojure, things are similar but simpler. The try and throw special forms give you all the capabilities of Java's try, catch, finally, and throw. But you should not have to use them very often, for the following reasons:

- You do not have to deal with checked exceptions in Clojure.
- You can use macros such as with-open to encapsulate resource cleanup.

Let's see what this looks like in practice.

Keeping Exception Handling Simple

Java programs often wrap checked exceptions at abstraction boundaries. A good example is Apache Ant, which tends to wrap low-level exceptions (such as I/O exceptions) with an Ant-level build exception:

```
// Ant-like code (simplified for clarity)
try {
    newManifest = new Manifest(r);
} catch (IOException e) {
    throw new BuildException(...);
}
```

In Clojure, you are not forced to deal with checked exceptions. You do not have to catch them or declare that you throw them. So, the previous code would translate to the following:

```
(Manifest. r)
```

The absence of exception wrappers makes idiomatic Clojure code easier to read, write, and maintain than idiomatic Java. That said, nothing prevents you from explicitly catching, wrapping, and rethrowing exceptions in Clojure. It simply is not required. You *should* catch exceptions when you plan to respond to them in a meaningful way.

Checked Exceptions

Java's checked exceptions must be explicitly caught or rethrown from every method where they can occur. This seemed like a good idea at first: checked exceptions could use the type system to rigorously document error handling, with compiler enforcement. Most Java programmers now consider checked exceptions a failed experiment, because their costs in code bloat and maintainability outweigh their advantages. For more on the history of checked exceptions, see Rod Waldhoff's article^a and the accompanying links.

a. <http://tinyurl.com/checked-exceptions-mistake>

Cleaning Up Resources

Garbage collection will clean up resources in memory. If you use resources that live outside of garbage-collected memory, such as file handles, you need to make sure that you clean them up, even in the event of an exception. In Java, this is normally handled in a `finally` block.

If the resource you need to free follows the convention of having a `close` method, you can use Clojure's `with-open` macro:

```
(with-open [name init-form] & body)
```

Internally, `with-open` creates a `try` block, sets `name` to the result of `init-form`, and then runs the forms in `body`. Most important, `with-open` always closes the object bound to `name` in a `finally` block.

A good example of `with-open` is the `spit` function in `clojure.string`:

```
(clojure.core/spit file content)
```

`spit` simply writes a string to file. Try it:

```
(spit "hello.out" "hello, world")
-> nil
```

You should now find a file at `hello.out` with the contents `hello, world`.

The implementation of `spit` is simple:

```
; from clojure.core
(defn spit
  "Opposite of slurp. Opens f with writer, writes content, then
  closes f. Options passed to clojure.java.io/writer."
  {:added "1.2"}
  [f content & options]
  (with-open [^java.io.Writer w (apply jio/writer f options)]
    (.write w (str content))))
```

spit creates a `PrintWriter` on `f`, which can be just about anything that is writable: a file, a URL, a URI, or any of Java's various writers or output streams. It then prints content to the writer. Finally, with-open guarantees that the writer is closed at the end of spit.

If you need to do something other than close in a finally block, the Clojure try form looks like this:

```
(try expr* catch-clause* finally-clause?)
; catch-clause -> (catch classname name expr*)
; finally-clause -> (finally expr*)
```

It can be used thusly:

```
(try
  (throw (Exception. "something failed"))
  (finally
    (println "we get to clean up")))
| we get to clean up
-> java.lang.Exception: something failed
```

The previous fragment also demonstrates Clojure's throw form, which simply throws whatever exception is passed to it.

Responding to an Exception

The most interesting case is when an exception handler attempts to respond to the problem in a catch block. As a simple example, consider writing a function to test whether a particular class is available at runtime:

```
src/examples/interop.clj
; not caller-friendly
(defn class-available? [class-name]
  (Class/forName class-name))
```

This approach is not very caller-friendly. The caller simply wants a yes/no answer but instead gets an exception:

```
(class-available? "borg.util.Assimilate")
-> java.lang.ClassNotFoundException: borg.util.Assimilate
```

A friendlier approach uses a catch block to return false:

```
src/examples/interop.clj
(defn class-available? [class-name]
  (try
    (Class/forName class-name) true
    (catch ClassNotFoundException _ false)))
```

The caller experience is much better now:

```
(class-available? "borg.util.Assimilate")
-> false
```

```
(class-available? "java.lang.String")
-> true
```

Clojure gives you everything you need to throw and catch exceptions and to cleanly release resources. At the same time, Clojure keeps exceptions in their place. They are important but not so important that your mainline code is dominated by the exceptional.

Clojure is designed to let you get things done and have fun while doing it. However, an important part of getting things done is being able to use your platform to its full potential. Throughout the rest of the book, we have looked at doing things the idiomatic Clojure way. In this chapter, we will do it Java style.

To provide the full power of the Java platform, Clojure does several things:

- Type hinting and inference, where needed, give the performance that people (incorrectly) associate with statically typed languages.
- Ahead-of-time (AOT) compilation lets Clojure programs participate in the binary-artifact-centric Java ecosystem as an equal player.
- While `reify`, `defrecord`, and `deftype` are preferable as flexible implementation tools, Clojure also provides interop forms that give you access to the ugly parts of Java interop.
- Clojure (and Clojure Contrib) have a rapidly expanding set of “batteries included” libraries for common tasks. (But, you can always call the raw Java libraries if these libraries do not include something you need.)
- If the techniques in this chapter seem ugly or unnecessary for the problem you are solving, that’s great! Run with it. But if you need to squeeze out that last bit of performance or play well with an ancient, ugly library, this chapter is for you.

9.2 Wrestling with the Integers

Clojure provides three different sets of operations for integer types:

- The unchecked operators
- The default operators
- The promoting operators

The following table gives a sampling of these operator types.

Unchecked	Default	Promoting
unchecked-add	+	+'
unchecked-subtract	-	-'
unchecked-multiply	*	*'
unchecked-inc	inc	inc'
unchecked-dec	dec	dec'

The unchecked operators correspond exactly with primitive math in Java. They are fast but terrifically dangerous in that they can overflow silently and give incorrect answers. In Clojure, the unchecked operators should be used only in the rare situation that overflow is the desired behavior or when performance is paramount and you are certain overflow is impossible or irrelevant.

```
(unchecked-add 9223372036854775807 1)
-> -9223372036854775808
```

The default operators use Java primitives where possible for performance but always make overflow checks and throw an exception.

```
(+ 9223372036854775807 1)
-> ArithmeticException integer overflow
```

The promoting operators will automatically promote from primitives to big numbers on overflow. This makes it possible to handle an arbitrary range but at significant performance cost. Because primitives and big numbers share no common base type, math with the promoting operators precludes the use of primitives as return types.

```
(+' 9223372036854775807 1)
-> 9223372036854775808N
```

Clojure relies on Java's `BigDecimal` class for arbitrary-precision decimal numbers. See the online documentation¹ for details. `BigDecimal`s provide arbitrary precision but at a price: `BigDecimal` math is significantly slower than Java's floating-point primitives.

Clojure has its own `BigInt` class to handle `BigInteger` conversions. Clojure's `BigInt` has some performance improvements over using Java's `BigInteger` directly. It also wraps some of the rough edges of `BigInteger`. In particular, it properly implements `hashCode`. This makes equality take precedence over representation, which you will see in almost every abstraction in the language.

1. <http://docs.oracle.com/javase/6/docs/api/java/math/BigDecimal.html>

Under the hood, Clojure uses Java's `BigInteger`. The performance difference comes in how `BigInteger` treats its values. A `BigInteger` consists of a `Long` part and a `BigInteger` part. When the value passed into a `BigInteger` is small enough to be treated as a `Long`, it is. When numerical operations are performed on `BigInts`, if their result is small enough to be treated as a `Long`, it is. This gives the user the ability to add the overflow hint (`N`) without paying the `BigInteger` cost until it is absolutely necessary.

9.3 Optimizing for Performance

In Clojure, it is idiomatic to call Java using the techniques described in [Section 2.5, *Calling Java*, on page 43](#). The resulting code will be fast enough for 90 percent of scenarios. When you need to, though, you can make localized changes to boost performance. These changes will not change how outside callers invoke your code, so you are free to make your code work and then make it fast.

Using Primitives for Performance

In the preceding sections, function parameters carry no type information. Clojure simply does the right thing. Depending on your perspective, this is either a strength or a weakness. It's a strength, because your code is clean and simple and can take advantage of duck typing. But it's also a weakness, because a reader of the code cannot be certain of datatypes and because doing the right thing carries some performance overhead.

Consider a function that calculates the sum of the numbers from 1 to `n`:

```
; performance demo only, don't write code like this
(defn sum-to [n] (loop [i 1 sum 0]
  (if (<= i n) (recur (inc i) (+ i sum)) sum)))
```

You can verify that this function works with a small input value:

```
(sum-to 10)
=> 55
```

Let's see how `sum-to` performs. To time an operation, you can use the `time` function. When benchmarking, you'll tend to want to take several measurements so that you can eliminate start-up overhead plus any outliers; therefore, you can call `time` from inside a `dotimes` macro:

```
(dotimes bindings & body)
```

`dotimes` will execute its body repeatedly, with the name bound to integers from zero to `n-1`. Using `dotimes`, you can collect five timings of `sum-to` as follows:

```
(dotimes [_ 5] (time (sum-to 10000)))
| "Elapsed time: 0.149 msecs"
| "Elapsed time: 0.126 msecs"
| "Elapsed time: 0.194 msecs"
| "Elapsed time: 0.279 msecs"
-> "Elapsed time: 0.212 msecs"
```

To speed things up, you can hint the argument and return type as long. Clojure's type inference will flow this hint to all the internal operations and function calls inside the function.

```
(defn ^long integer-sum-to [^long n]
  (loop [i 1 sum 0]
    (if (<= i n)
      (recur (inc i) (+ i sum))
      sum))))
```

The integer-sum-to is indeed faster:

```
(dotimes [_ 5] (time (integer-sum-to 10000)))
| "Elapsed time: 0.044 msecs"
| "Elapsed time: 0.023 msecs"
| "Elapsed time: 0.025 msecs"
| "Elapsed time: 0.023 msecs"
-> "Elapsed time: 0.02 msecs"
```

Clojure's primitive math is still correct, in that it will check for overflow and throw an exception. Is that as fast as things can get? Java programmers have access to super-fast busted math: arithmetic operations that have the maximum possible performance but can silently overflow and corrupt data.

Clojure provides access to Java's arithmetic semantics through the unchecked family of functions. Maybe you can get an even faster function by using the unchecked version of +, unchecked-add:

```
(defn ^long unchecked-sum-to [^long n]
  (loop [i 1 sum 0]
    (if (<= i n)
      (recur (inc i) (unchecked-add i sum))
      sum))))
```

The unchecked-sum-to is not significantly faster:

```
(dotimes [_ 5] (time (unchecked-sum-to 10000)))
| "Elapsed time: 0.039 msecs"
| "Elapsed time: 0.018 msecs"
| "Elapsed time: 0.014 msecs"
| "Elapsed time: 0.015 msecs"
-> "Elapsed time: 0.015 msecs"
```

Orders of magnitude are important! Primitive hinting can make certain operations significantly faster. However, switching to Java's unchecked semantics is generally a losing proposition. You get a trivial performance gain on average, with the possibility of data corruption tomorrow.

So, why does Clojure provide these operations at all? Two reasons:

- Sometimes you actually want Java semantics. The primary use case for the unchecked operations is when you need to interoperate with other libraries that expect this behavior.
- Without trying them, nobody would know (or believe) that they weren't faster.

Prefer accuracy first and then optimize for speed only where necessary. `integer-sum-to` will throw an exception on overflow. This is bad, but the problem is easily detected:

```
(integer-sum-to 10000000000)
-> java.lang.ArithmeticException: integer overflow
```

`unchecked-sum-to` will fail silently on overflow. In a program setting, it can quietly but catastrophically corrupt data:

```
(unchecked-sum-to 10000000000)
-> -5340232216128654848 ; WRONG!!
```

Given the competing concerns of correctness and performance, you should normally prefer simple, undecorated code such as the original `sum-to`. If profiling identifies a bottleneck, you can force Clojure to use a primitive type in just the places that need it.

The `sum-to` example is deliberately simple in order to demonstrate the various options for integer math in Clojure. In a real Clojure program, it would be more expressive to implement `sum-to` using `reduce`. Summing a sequence is the same as summing the first two items, adding that result to the next item, and so on. That is exactly the loop that `(reduce + ...)` provides. With `reduce`, you can rewrite `sum-to` as a one-liner:

```
(defn better-sum-to [n]
  (reduce + (range 1 (inc n))))
```

The example also demonstrates an even more general point: pick the right algorithm to begin with. The sum of numbers from 1 to `n` can be calculated directly as follows:

```
(defn best-sum-to [n]
  (/ (* n (inc n)) 2))
```

Even without performance hints, this is faster than implementations based on repeated addition:

```
(dotimes [_ 5] (time (best-sum-to 10000)))
| "Elapsed time: 0.029 msecs"
| "Elapsed time: 0.0040 msecs"
| "Elapsed time: 0.0040 msecs"
| "Elapsed time: 0.0040 msecs"
-> "Elapsed time: 0.0030 msecs"
```

Performance is a tricky subject. Don't write ugly code in search of speed. Start by choosing appropriate algorithms and getting your code to work correctly. If you have performance issues, profile to identify the problems. Then, introduce only as much complexity as you need to solve those problems.

Adding Type Hints

Clojure supports adding type hints to function parameters, let bindings, variable names, and expressions. These type hints serve three purposes:

- Optimizing critical performance paths
- Documenting the required type
- Enforcing the required type at runtime

For example, consider the following function, which returns information about a Java class:

```
(defn describe-class [c]
  {:name (.getName c)
   :final (java.lang.reflect.Modifier/isFinal (.getModifiers c))})
```

You can ask Clojure how much type information it can infer, by setting the special variable `*warn-on-reflection*` to true:

```
(set! *warn-on-reflection* true)
-> true
```

The exclamation point on the end of `set!` is an idiomatic indication that `set!` changes mutable state. `set!` is described in detail in [Working with Java Callback APIs, on page 130](#). With `*warn-on-reflection*` set to true, compiling `describe-class` will produce the following warnings:

```
Reflection warning, line: 87
- reference to field getName can't be resolved.

Reflection warning, line: 88
- reference to field getModifiers can't be resolved.
```

These warnings indicate that Clojure has no way of knowing the type of `c`. You can provide a type hint to fix this, using the metadata syntax `^Class`:

```
(defn describe-class [^Class c]
  {:name (.getName c)
   :final (java.lang.reflect.Modifier/isFinal (.getModifiers c))})
```

With the type hint in place, the reflection warnings will disappear. The compiled Clojure code will be exactly the same as compiled Java code. Further, attempts to call `describe-class` with something other than a `Class` will fail with a `ClassCastException`:

```
(describe-class StringBuffer)
{:name "java.lang.StringBuffer", :final true}

(describe-class "foo")
-> java.lang.ClassCastException: \
    java.lang.String cannot be cast to java.lang.Class
```

If your `ClassCastException` provides a less helpful error message, it is because you are using a version of Java prior to Java 6. Improved error reporting is one of many good reasons to run your Clojure code on Java 6 or newer.

When you provide a type hint, Clojure will insert an appropriate class cast in order to avoid making slow, reflective calls to Java methods. But if your function does not actually call any Java methods on a hinted object, then Clojure will not insert a cast. Consider this `wants-a-string` function:

```
(defn wants-a-string [^String s] (println s))
-> #'user/wants-a-string
```

You might expect that `wants-a-string` would complain about nonstring arguments. In fact, it will be perfectly happy:

```
(wants-a-string "foo")
-> foo

(wants-a-string 0)
-> 0
```

Clojure can tell that `wants-a-string` never actually uses its argument as a string (`println` will happily try to print any kind of argument). Since no string methods need to be called, Clojure does not attempt to cast `s` to a string.

When you need speed, type hints will let Clojure code compile down to the same code Java will produce. But you won't need type hints that often. Make your code right first, and then worry about making it fast.

9.4 Creating Java Classes in Clojure

Clojure's objects all implement reasonable Java interfaces:

- Clojure's data structures implement interfaces from the Java Collections API.
- Clojure's functions implement Runnable and Callable.

In addition to these generic interfaces, you will occasionally need domain-specific interfaces. Often this comes in the form of callback handlers for event-driven APIs such as Swing or some XML parsers. Clojure can easily generate one-off proxies or classes on disk when needed, using a fraction of the lines of code necessary in Java.

Creating Java Proxies

To interoperate with Java, you will often need to implement Java interfaces. A good example is parsing XML with a Simple API for XML (SAX) parser. To get ready for this example, go ahead and import the following classes. We'll need them all before we are done:

```
(import '(org.xml.sax InputSource)
        '(org.xml.sax.helpers DefaultHandler)
        '(java.io StringReader)
        '(javax.xml.parsers SAXParserFactory))
```

To use a SAX parser, you need to implement a callback mechanism. The easiest way is often to extend the DefaultHandler class. In Clojure, you can extend a class with the proxy function:

```
(proxy class-and-interfaces super-cons-args & fns)
```

As a simple example, use proxy to create a DefaultHandler that prints the details of all calls to startElement:

```
(def print-element-handler
  (proxy [DefaultHandler] []
    (startElement [uri local qname atts]
      (println (format "Saw element: %s" qname)))))
```

proxy generates an instance of a proxy class. The first argument to proxy is [DefaultHandler], a vector of the superclass and superinterfaces. The second argument, [], is a vector of arguments to the base class constructor. In this case, no arguments are needed.

After the proxy setup comes the implementation code for zero or more proxy methods. The proxy shown earlier has one method. Its name is startElement, and it takes four arguments and prints the name of the qname argument.

Now all you need is a parser to pass the handler to. This requires plowing through a pile of Java factory methods and constructors. For a simple exploration at the REPL, you can create a function that parses XML in a string:

```
(defn demo-sax-parse [source handler]
  (.. SAXParserFactory newInstance newSAXParser
      (parse (InputSource. (StringReader. source)) handler)))
```

Now the parse is easy:

```
(demo-sax-parse "<foo>
  <bar>Body of bar</bar>
</foo>" print-element-handler)
| Saw element: foo
| Saw element: bar
```

The previous example demonstrates the mechanics of creating a Clojure proxy to deal with Java's XML interfaces. You can take a similar approach to implementing your own custom Java interfaces. But if all you are doing is XML processing, the `clojure.data.xml` library already has terrific XML support and can work with any SAX-compatible Java parser.

The proxy mechanism is completely general and can be used to generate any kind of Java object you want, on the fly. Sometimes the objects are so simple you can fit the entire object in a single line. The following code creates a new thread and then creates a new dynamic subclass of `Runnable` to run on the new thread:

```
(.start (Thread.
  (proxy [Runnable] [] (run [] (println "I ran!")))))
```

In Java, you must provide an implementation of every method on every interface you implement. In Clojure, you can leave them out:

```
(proxy [Callable] []) ; proxy with no methods (??)
```

If you omit a method implementation, Clojure provides a default implementation that throws an `UnsupportedOperationException`:

```
(.call (proxy [Callable] []))
-> java.lang.UnsupportedOperationException: call
```

The default implementation does not make much sense for interfaces with only one method, such as `Runnable` and `Callable`, but it can be handy when you are implementing larger interfaces and don't care about some of the methods.

So far in this section, you have seen how to use proxy to create implementations of Java interfaces. This is very powerful when you need it, but often Clojure

is already there on your behalf. For example, functions automatically implement `Runnable` and `Callable`:

```
; normal usage: call an anonymous function
(#(println "foo"))
foo
; call through Runnable's run
(.run #(println "foo"))
foo
; call through Callable's call
(.call #(println "foo"))
foo
```

This makes it very easy to pass Clojure functions to other threads:

```
(dotimes [i 5]
  (.start
    (Thread.
      (fn []
        (Thread/sleep (rand 500))
        (println (format "Finished %d on %s" i (Thread/currentThread)))))))
```

For one-off tasks such as XML and thread callbacks, Clojure's proxies are quick and easy to use. If you need a longer-lived class, you can generate new named classes from Clojure as well.

Using Java Collections

Clojure's collections supplant the Java collections for most purposes. Clojure's collections are concurrency-safe, have good performance characteristics, and implement the appropriate Java collection interfaces. So, you should generally prefer Clojure's own collections when you are working in Clojure and even pass them back into Java when convenient.

If you do choose to use the Java collections, nothing in Clojure will stop you. From Clojure's perspective, the Java collections are classes like any other, and all the various Java interop forms will work. But the Java collections are designed for lock-based concurrency. They will not provide the concurrency guarantees that Clojure collections do and will not work well with Clojure's software transactional memory.

One place where you will need to deal with Java collections is the special case of Java arrays. In Java, arrays have their own syntax and their own bytecode instructions. Java arrays do not implement any Java interface. Clojure collections cannot masquerade as arrays. (Java collections can't either!) The Java platform makes arrays a special case in every way, so Clojure does too.

Clojure provides `make-array` to create Java arrays:

```
(make-array class length)
(make-array class dim & more-dims)
```

make-array takes a class and a variable number of array dimensions. For a one-dimensional array of strings, you might say this:

```
(make-array String 5)
-> #<String[] [Ljava.lang.String;@45a270b2>
```

The odd output is courtesy of Java's implementation of toString() for arrays: [Ljava.lang.String; is the JVM specification's encoding for "one-dimensional array of strings." That's not very useful at the REPL, so you can use Clojure's seq to wrap any Java array as a Clojure sequence so that the REPL can print the individual array entries:

```
(seq (make-array String 5))
-> (nil nil nil nil nil)
```

Clojure also includes a family of functions with names such as int-array for creating arrays of Java primitives. You can issue the following command at the REPL to review the documentation for these and other array functions:

```
(find-doc "-array")
```

Clojure provides a set of low-level operations on Java arrays, including aset, aget, and alength:

```
(aset java-array index value)
(aset java-array index-dim1 index-dim2 ... value)
(aget java-array index)
(aget java-array index-dim1 index-dim2 ...)
(alength java-array)
```

Use make-array to create an array, and then experiment with using aset, aget, and alength to work with the array:

```
(defn painstakingly-create-array []
  (let [arr (make-array String 5)]
    (aset arr 0 "Painstaking")
    (aset arr 1 "to")
    (aset arr 2 "fill")
    (aset arr 3 "in")
    (aset arr 4 "arrays")
    arr))

(aget (painstakingly-create-array) 0)
-> "Painstaking"

(alength (painstakingly-create-array))
-> 5
```

Most of the time, you will find it simpler to use higher-level functions such as `to-array`, which creates an array directly from any collection:

(to-array sequence)

`to-array` always creates an Object array:

```
(to-array ["Easier" "array" "creation"])
-> #<Object[] [Ljava.lang.Object;@1639f9e3>
```

`to-array` is also useful for calling Java methods that take a variable argument list, such as `String/format`:

```
; example. prefer clojure.core/format (String/format "Training Week: %s Mileage: %d"
(String/format "Training Week: %s Mileage: %d"
  (to-array [2 26])))
-> "Training Week: 2 Mileage: 26"
```

`to-array`'s cousin `into-array` can create an array with a more specific type than `Object`.

```
(into-array type? seq)
```

You can pass an explicit type as an optional first argument to `into-array`:

```
(into-array String ["Easier", "array", "creation"])
-> #<String[] [Ljava.lang.String;@391ecf28>
```

If you omit the type argument, `into-array` will guess the type based on the first item in the sequence:

```
(into-array ["Easier" "array" "creation"])
-> #<String[] [Ljava.lang.String;@76bfd849>
```

As you can see, the array contains `Strings`, not `Objects`. If you want to transform every element of a Java array without converting to a Clojure sequence, you can use `amap`:

(amap a idx ret expr)

`amap` will create a clone of the array `a`, binding that clone to the name you specify in `ret`. It will then execute `expr` once for each element in `a`, with `idx` bound to the index of the element. Finally, `amap` returns the cloned array. You could use `amap` to uppercase every string in an array of strings:

```
(def strings (into-array ["some" "strings" "here"]))
-> #'user/strings

(seq (amap strings idx _ (.toUpperCase (aget strings idx))))
-> ("SOME" "STRINGS" "HERE")
```

The `ret` parameter is set to `_` to indicate that it is not needed in the map expression, and the wrapping `seq` is simply for convenience in printing the result at the REPL. Similar to `amap` is `areduce`:

```
(areduce a idx ret init expr)
```

Where `amap` produces a new array, `areduce` produces anything you want. The `ret` is initially set to `init` and later set to the return value of each subsequent invocation of `expr`. `areduce` is normally used to write functions that “tally up” a collection in some way. For example, the following call finds the length of the longest string in the `strings` array:

```
(areduce strings idx ret 0 (max ret (.length (aget strings idx))))
-> 7
```

`amap` and `areduce` are special-purpose macros for interoperating with Java arrays.

9.5 A Real-World Example

While it’s great to talk about the different interop cases and learn how to eke out some additional performance using Java’s primitive forms, you still need to have some practical, hands-on knowledge. In this example, we will be building an application to test the availability of websites. The goal here is to check to see whether the website returns an HTTP 200 OK response. If anything other than our expected response is received, it should be marked as unavailable.

Again, we will use the Leiningen build tool. Refer to [Section 1.2, Clojure Coding Quick Start, on page 11](#) if you don’t have Leiningen installed already. Let’s start by creating a new project:

```
lein new pinger
```

Open your `project.clj` file and modify the contents to match what we are going to be working on. Be sure to update Clojure to the latest version:

```
(defproject pinger "0.0.1-SNAPSHOT"
  :description "A website availability tester"
  :dependencies [[org.clojure/clojure 1.3.0]])
```

Grab the dependencies by running `lein deps`:

```
lein deps
```

First we need to write the code that connects to a URL and captures the response code. We can accomplish this by using Java’s URL class.

```
(ns pinger.core
  (:import (java.net URL HttpURLConnection)))

(defn response-code [address]
  (let [conn ^HttpURLConnection (.openConnection (URL. address))
        code (.getResponseCode conn)]
    (when (< code 400)
      (-> conn .getInputStream .close))
    code))
```

Give it a try in the REPL:

```
(response-code "http://google.com")
-> 200
```

Now let's create a function that uses `response-code` and decides whether the specified URL is available. We will define `available` in our context as returning an HTTP 200 response code.

```
(defn available? [address]
  (= 200 (response-code address)))

(available? "http://google.com")
-> true

(available? "http://google.com/badurl")
-> false
```

Next we need a way to start our program and have it check a list of URLs that we care about every so often and report their availability. Let's create a `-main` function.

```
(defn -main []
  (let [addresses '("http://google.com"
                    "http://amazon.com"
                    "http://google.com/badurl")]
    (while true
      (doseq [address addresses]
        (println (available? address)))
      (Thread/sleep (* 1000 60)))))
```

In this example, we create a list of addresses (two good and one bad) and use a simple while loop that never exits to obtain a never-ending program execution. It will continue to check these URLs once a minute until the program is terminated. Since we are exporting a `-main` function, don't forget to add `:gen-class` to your namespace declaration.

```
(ns pinger.core
  (:import (java.net URL))
  (:gen-class))
```

Now that we have the fundamentals in place, we need to tell Leningen where our main function is located. Open `project.clj` and add the `:main` declaration:

```
(defproject pinger "0.0.1-SNAPSHOT"
  :description "A website availability tester"
  :dependencies [[org.clojure/clojure "1.3.0"]]
  :main pinger.core)
```

It's time to compile our program into a JAR file and run it. To do this, run the following:

```
lein uberjar
java -jar pinger-0.0.1-SNAPSHOT-standalone.jar
true
false
true
```

You should see your program start and continue to run until you press Ctrl-C to stop it.

Adding Real Continuous Loop Behavior

A while loop that is always true will continue to run until terminated, but it's not really the cleanest way to obtain the result because it doesn't allow for a clean shutdown. We can use a scheduled thread pool that will start and execute the desired command in a similar fashion as the while loop but with a much greater level of control. Create a file in the `src` directory called `scheduler.clj` and enter the following code:

```
(ns pinger.scheduler
  (:import (java.util.concurrent ThreadPoolExecutor
    ScheduledThreadPoolExecutor TimeUnit)))
(defn scheduled-executor
  "Create a scheduled executor."
  ^ScheduledThreadPoolExecutor [threads]
  (ScheduledThreadPoolExecutor. threads))

(defn periodically
  "Schedules function f to run on executor e every 'delay'
  milliseconds after a delay of 'initial-delay' Returns
  a ScheduledFuture."
  ^ScheduledFuture [e f & {:keys [initial-delay delay]}]
  (.scheduleWithFixedDelay
    e f
    initial-delay delay
    TimeUnit/MILLISECONDS))
(defn shutdown-executor
  "Shutdown an executor."
  [^ThreadPoolExecutor e]
  (.shutdown e))
```

This namespace provides functions to create and shut down a Java ScheduledExecutor. It also defines a function called `periodically` that will accept an executor, a function, an initial-delay, and a repeated delay. It will execute the function for the first time after the initial delay and then continue to execute the function with the delay specified thereafter. This will continue to run until the thread pool is shut down.

Let's update `pinger.core` to take advantage of the scheduling code as well as make the `-main` function responsible only for calling a function that starts the loop. Replace the old `-main` with the following functions:

```
(defn check []
  (let [addresses '("http://google.com"
                    "http://google.com/404"
                    "http://amazon.com")]
    (doseq [address addresses]
      (println (available? address)))))

(def immediately 0)
(def every-minute (* 60 1000))

(defn start [e]
  "REPL helper. Start pinger on executor e."
  (scheduler/periodically e check
                           :initial-delay immediately
                           :delay every-minute))

(defn stop [e]
  "REPL helper. Stop executor e."
  (scheduler/shutdown-executor e))

(defn -main []
  (start (scheduler/scheduled-executor 1)))
```

Make sure to update your namespace declaration to include the scheduler code:

```
(ns pinger.core
  (:import (java.net URL))
  (:require [pinger.scheduler :as scheduler])
  (:gen-class))
```

Not everything in the previous sample is necessary, but it makes for more readable code. Adding the `start` and `stop` functions makes it easy to work interactively from the REPL, which will be a huge advantage should you choose to extend this example. Give everything one last check by running `lein uberjar` and executing the JAR file. The program should function exactly as it did before.

Logging

So far, we have produced a program capable of periodically checking the availability of a list of websites. However, it lacks the ability to keep track of what it has done and to notify us when a site is unavailable. We can solve both of these issues with logging. There are a lot of logging options for Java applications, but for this example we will use log4j. It gives us a real logger to use, and it gives us an email notification. This is great because we will have the ability to send email alerts when a website isn't available. To do this, we will need to pull the log4j and mail libraries into our application. To make it easier to take advantage of log4j, we will also pull in clojure.tools.logging. Open your project.clj file and add clojure.tools.logging, log4j, and mail:

```
(defproject pinger "0.0.1-SNAPSHOT"
  :description "A website availability tester"
  :dependencies [[org.clojure/clojure "1.3.0"]
                 [org.clojure/tools.logging "0.2.3"]
                 [log4j "1.2.16"]
                 [javax.mail/mail "1.4.1"]]
  :main pinger.core)
```

Also pull the dependencies in with Leiningen:

```
lein deps
```

The great part about the Clojure logging library is that it will use any standard Java logging library that is on the classpath, so there is no additional wiring required between log4j and your application. Create a folder in the root of your project called resources. Leiningen automatically adds the contents of this folder to the classpath, and you will need that for your log4j properties file. Create a file under the resources directory named log4j.properties and add the following contents:

```
log4j.rootLogger=info, R, email
log4j.appender.R=org.apache.log4j.RollingFileAppender
log4j.appender.R.File=logs/pinger.log
log4j.appender.R.MaxFileSize=1000KB
log4j.appender.R.MaxBackupIndex=1
log4j.appender.R.layout=org.apache.log4j.PatternLayout
log4j.appender.R.layout.ConversionPattern=%d{ISO8601} %-5p [%c] - %m%n
log4j.appender.email=org.apache.log4j.net.SMTPAppender
log4j.appender.email.SMTPHost=localhost
log4j.appender.email.From=system@yourapp.com
log4j.appender.email.To=recipient@yourapp.com
log4j.appender.email.Subject=[Pinger Notification] - Website Down
log4j.appender.email.threshold=error
log4j.appender.email.layout=org.apache.log4j.PatternLayout
log4j.appender.email.layout.conversionPattern=%d{ISO8601} %-5p [%c] - %m%n
```

This sets up standard logging to `pinger.log` and will send an email notification for anything logged to the error log level, which in our case is when a website doesn't respond with an HTTP 200 response or when an exception is thrown while checking the site. Make sure to change the email information to something that works in your environment.

Let's update the code and add logging. The goal here is to replace any `println` statements with log messages. Open `core.clj`, add the `info` and `error` functions from `clojure.tools.logging` into your namespace declaration, and create a function to record the results.

```
(ns pinger.core
  (:import (java.net URL))
  (:require [pinger.scheduler :as scheduler]
            [clojure.tools.logging :as logger]))
(gen-class)
...

(defn record-availability [address]
  (if (available? address)
    (logger/info (str address " is responding normally")))
    (logger/error (str address " is not available"))))
```

Also update `check` to reflect the changes:

```
(defn check []
  (let [addresses '("http://google.com"
                    "http://google.com/404"
                    "http://amazon.com")]
    (doseq [address addresses]
      (record-availability address))))
```

Rebuild to try your program again. You should notice a newly created logs directory that you can check for program execution. You should also notice an email come in with an error message. If you get a “connection refused” error on port 25, you will need to set up a mail transport agent on your machine to enable mail sending. You now have a way to notify people of a website failure!

Configuration

We have hard-coded our list of websites to monitor, and that simply won't work! We need a way to give a list of sites to monitor from some external source. We could use a properties file, database, or web service to accomplish this. For ease of explanation, we will go with a properties file. Create a file named `pinger.properties` in the root directory of the application and add the following to it:

```
urls=http://google.com,http://amazon.com,http://google.com/badurl
```

We need a way to load this file and create a collection of sites to feed into the check function. Create a file named `config.clj` in the `src` directory:

```
(ns pinger.config
  (:use [clojure.java.io :only (reader resource)])
  (:require [clojure.string :as str])
  (:import (java.util Properties)))

(defn load-properties [src]
  (with-open [rdr (reader src)]
    (doto (Properties.)
      (.load rdr))))

(defn config
  []
  (load-properties (resource "pinger.properties")))
```

As long as `pinger.properties` is on the classpath, the `config` function will read `pinger.properties` into a Java properties object. All we have left to do is get the `urls` attribute and put it into a list. Add the following function into the `config` namespace:

```
(defn urls [conf]
  (str/split (.get conf "urls") #","))
```

Finally, update the `check` function in `core.clj` to use the new configuration function.

```
(ns pinger.core
  (:import (java.net URL))
  (:require [pinger.scheduler :as scheduler]
            [clojure.tools.logging :as logger]
            [pinger.config :as config])
  (:gen-class))

...

(defn check []
  (doseq [address (config/urls (config/config))]
    (record-availability address)))
```

Rebuild your application with Leiningen and try it. Remember to put the root directory on the classpath so that the application can find `pinger.properties`.

```
java -cp .:pinger-0.0.1-standalone.jar pinger.core
```

We now have what we need to succeed. In this example, we covered the following:

- Using Java's URL to check a website to see whether it was available
- Using Java's ScheduledThreadPoolExecutor to create a periodically running task
- Using log4j with clojure.tools.logging to send error notifications
- Using Java's property system for configuration
- Using Leiningen to create stand-alone executable JAR files

We could do quite a few things to expand this example. We could redefine what it means for a website to be available by adding requirements for certain HTML elements to be present or for the response to return in a certain time to cover an SLA. Try adding to this example and see what you can come up with.

9.6 Wrapping Up

We just covered a good chunk of how Clojure and Java get along. We even mixed the two up in some interesting ways. Since we have started to experiment with things outside of the Clojure language, it is probably a good time to start talking about different libraries you can use to build real-world Clojure applications. We will do just that in the next chapter.